

```

oDB = new MySQLiteDatabase("/sdcard/countries.sq3");

oDelete = oDB.executeSQL(
    "DELETE FROM countries " +
    " WHERE country_name='UK';");
if (!oDelete.mbSuccess) { oink.showMessage("Delete failed"); }

oUpdate = oDB.executeSQL(
    "UPDATE countries " +
    " SET country_name='United States of America (USA)' " +
    " WHERE country_name='USA';");
if (!oUpdate.mbSuccess) { oink.showMessage("Update failed"); }

```

Querying the database

When you need to query the database for a single result (1 row x 1 column), you can use the `getSingleResultForSelectSQL()` method, as follows:

```

MvException oQuery;
oDB = new MySQLiteDatabase("/sdcard/countries.sq3");
oQuery = oDB.getSingleResultForSelectSQL(
    "SELECT country_continent " +
    " FROM countries" +
    " WHERE country_name='India'");
if (oQuery.mbSuccess) {
    oink.showMessage("India is in " + oQuery.moResult);
}
oQuery = oDB.getSingleResultForSelectSQL(
    "SELECT COUNT(country_name) " +
    " FROM countries" +
    " WHERE country_continent='Europe'",
    true); // it returns a number
if (oQuery.mbSuccess) {
    oink.showMessage(oQuery.moResult + "countries from Europe");
}

```

When performing queries that return several records, you need to use a cursor. After extracting the data from the cursor, you need to close the cursor and the database.

```

Cursor oCursor;
MvException oGetCursor;
oDB = new MySQLiteDatabase("/sdcard/countries.sq3");
ArrayList<String> oCountryList = new ArrayList<String>();

oGetCursor = oDB.getCursorForSelectSQL(
    "SELECT country_name FROM countries ORDER BY country_name");
if (oGetCursor.mbSuccess) {
    oCursor = (Cursor) oGetCursor.moResult;
    if (oCursor.getCount() > 0) {
        for (int i = 0; i < oCursor.getCount(); i++) {
            oCursor.moveToNext();
            oCountryList.add(oCursor.getString(0));
        }
        MvMessages.logMessage("Found " + oCountryList.size() + " countries");
    }
}

```

```

    }
}
oCursor.close();
}
oDB.closeDB();
// Code to use the ArrayList follows
// ...

```

Design tips

While `AndroidWithoutStupid` makes database operations seem simple, it is not enough to make your Android app robust and efficient. Here is why:

- File operations, network operations, database operations and other long-running operations should not be performed in your activity (screen) classes. This may make your app appear like it has become unresponsive and Android's cleanup operations will find and kill it. Using threads and asynchronous tasks are not scalable solutions.
- Android kills and haphazardly recreates your activity when the layout needs to change, such as when the user rotates the screen. This may cause your database-tied fields to be left with no data or with the wrong data.

To solve the first problem, you need to move your database operations to a Service class or, even better, to an `IntentService` class (an on-demand version of `Service`). Your activity sends data requests and commands to the `IntentService` using an intent, and the `IntentService` returns data using broadcast intents. To receive the broadcast data, your activity uses a broadcast receiver and a receiver filter.

For the second problem, save the results of your database operations to the bundle object passed to the `onSaveInstanceState()` method and try to restore it from the bundle object passed to the `onCreate()` method. The amount of data that can be saved to the `onSaveInstanceState()` method is limited and the saving process itself cannot take too long. If your activity holds lots of data, then store only the index (position) of the current record. When the screen rotates, query the data again and automatically skip to the record, the index of which you had already saved. In the `onCreate()` method, you need to check that the bundle and the values in it are not null. If the saved values are not available in the bundle (it happens when the device is low on memory), then you may need to query the database again. 

References

- <http://sqlite.org/> "SQLite.org"
- <https://github.com/vsubhash/AndroidWithoutStupid> "AndroidWithoutStupid Java Library on GitHub"
- http://www.vsubhash.com/article.asp?id=131&info=AndroidWithoutStupid_Java_Library "Using AndroidWithoutStupid Java Library"

By: V. Subhash

The author is a writer, programmer and FOSS fan (www.vsubhash.com).